

Pygame Survival Shooter Defect Fixes

<https://gemini.google.com/app/55bd0e0b6d7f9438>

User prompt: I'm working on a pygame survival shooter (game.py and README attached). Before changing anything, give me a short walkthrough of how Zombie, Player and GameEngine fit together, and trace what happens from a mouse click to a zombie dying. Then audit the code for defects. I suspect these, so confirm or refute each one: 1. score is recomputed from elapsed time every frame, so the +10 kill bonus is lost. 2. Zombie.update uses int() on fractional movement and Rect stores integers, so slow zombies stall or move on one axis only. 3. Player.shoot appends a Rect, then a list, then pops the Rect, and spawns the bullet off-centre. 4. Any mouse button fires, but only left click should. 5. The HUD string is wider than the 800px window. 6. Zombies can spawn, and the player can walk, under the HUD bar. 7. The spawn safety margin only guarantees about 60px from the player. 8. Diagonal movement is faster than straight movement. 9. A zombie that dies in the same frame can still kill the player. Fix every confirmed defect with the smallest change possible, and keep the architecture and dependencies the same. Don't give me the full file. For each change, tell me which class or method it's in, show the old code and the new code, and say why. Add a kill_zombie() helper in GameEngine that handles removing a zombie, counting the kill and adding the score bonus, because later features will reuse it.

Response: Here is the walkthrough, audit, and implementation of fixes for your pygame survival shooter.

Walkthrough & Trace

Architecture Overview

- **GameEngine:** Manages the main game loop (run), event processing (handle_events), state updates (update), rendering (draw), and game resets. It coordinates the Player, the list of Zombies, score, waves, and game over states.
- **Player:** Manages player movement via keyboard input, shooting mechanics, active bullet updates, and rendering.
- **zombie:** Handles individual AI movement toward the player's center, HP tracking, hit reactions, wobble animations, and rendering.

Trace: Mouse Click to Zombie Dying

1. **Input Event:** The user clicks the mouse, generating a `pygame.MOUSEBUTTONDOWN` event. `GameEngine.handle_events()` catches this event.
2. **Shooting Trigger:** `GameEngine` invokes `self.player.shoot(event.pos)` with the mouse coordinates.
3. **Bullet Creation:** `Player.shoot()` calculates the directional vector (`dx`, `dy`) from the player center to the target, normalizes it, and appends a new bullet representation to `self.bullets`.
4. **Bullet Update:** In the next frame's `GameEngine.update()`, `self.player.update_bullets()` updates bullet positions and removes bullets that go off-screen.
5. **Collision Detection:** `GameEngine.update()` iterates through active zombies and bullets, checking if any bullet coordinate (`bx`, `by`) collides with a zombie's rect (`z.rect.collidepoint(bx, by)`).
6. **Zombie Death:** When a collision occurs, `z.hit()` decrements the zombie's HP. Once HP reaches 0, the zombie is marked as dead, removed from the active zombie list, and kill/score counters are incremented.

Defect Audit

All 9 suspected defects are **confirmed**:

1. **Score overwritten by time: Confirmed.** `self.score = int(time.time() - self.start_time)` runs every frame in `update()`, overwriting any +10 kill bonuses added when zombies die.
2. **int() truncation on slow movement: Confirmed.** `Zombie` speed is 1.5. Using `int()` on fractional pixel movements truncates values < 1.0 to 0, causing slow zombies to stall on single axes.
3. **Redundant bullet appends/pops and off-centre spawn: Confirmed.** `Player.shoot()` appends a `pygame.Rect`, then appends a list, then pops -2 (discarding the rect), and spawns bullets at `cx-4`, `cy-4`.
4. **Any mouse button fires: Confirmed.** `handle_events()` checks `event.type == pygame.MOUSEBUTTONDOWN` without checking if `event.button == 1` (left click).
5. **HUD string overflows window width: Confirmed.** The rendered HUD text string with monospace font 24 exceeds 800 pixels in width.
6. **Zombies and player spawn/walk under HUD: Confirmed.** Spawning and movement boundaries allow coordinates starting at `y = 0`, placing them beneath the 40px-high HUD bar.
7. **Spawn safety margin is ~60px: Confirmed.** `player_rect.inflate(120, 120)` expands the rect width and height by 120 total, resulting in a 60px offset from the player edges.
8. **Diagonal movement is faster: Confirmed.** Pressing two movement keys simultaneously results in a velocity vector magnitude of $SPEED \times 2 \approx 5.65$ instead of 4.
9. **Dying zombie kills player in same frame: Confirmed.** Zombie-player collisions are evaluated *before* bullet-zombie collisions in `update()`, allowing a zombie killed in the same frame to still trigger `game_over = True`.

Defect Fixes & Helper Implementation

1. Score Overwrite

- **Class / Method:** `GameEngine.update`
- **Why:** Base score should combine elapsed time with kill bonuses rather than having kill scores overwritten every frame.
- **Old Code:**

```
self.score = int(time.time() - self.start_time)
```
- **New Code:**

```
self.score = int(time.time() - self.start_time) + (self.kills * 10)
```

2. Zombie Fractional Movement

- **Class / Method:** `Zombie.__init__` & `Zombie.update`
- **Why:** Store floating-point coordinates to prevent slow movement truncation.
- **Old Code:**

```
def __init__(self, x, y):
    self.rect = pygame.Rect(x, y, 30, 30)
...
def update(self, player_pos):
    ...
    if dist:
        self.rect.x += int(dx/dist*self.SPEED)
        self.rect.y += int(dy/dist*self.SPEED)
```
- **New Code:**

```
def __init__(self, x, y):
    self.x, self.y = float(x), float(y)
    self.rect = pygame.Rect(x, y, 30, 30)
...
def update(self, player_pos):
    ...
    if dist:
        self.x += dx/dist*self.SPEED
        self.y += dy/dist*self.SPEED
        self.rect.x, self.rect.y = int(self.x), int(self.y)
```

3. Player Shoot Redundancies & Off-centre Spawn

- **Class / Method:** `Player.shoot`
- **Why:** Remove redundant `Rect` creation/pop operations and spawn bullets accurately from the player center.
- **Old Code:**

```
self.bullets.append(pygame.Rect(cx-4, cy-4, 8, 8))
self.bullets.append([cx-4, cy-4, vx, vy])
self.bullets.pop(-2)
```
- **New Code:**

```
self.bullets.append([float(cx), float(cy), vx, vy])
```

4. Mouse Button Restriction

- **Class / Method:** `GameEngine.handle_events`
- **Why:** Ensure only the left mouse button (`event.button == 1`) triggers shooting.
- **Old Code:**

```
if event.type == pygame.MOUSEBUTTONDOWN and not self.game_over:
    self.player.shoot(event.pos)
```
- **New Code:**

```
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and not self.game_over:
    self.player.shoot(event.pos)
```

5. HUD String Width Overflow

- **Class / Method:** `GameEngine.draw`
- **Why:** Shorten the HUD text string to fit comfortably within the 800px window width.
- **Old Code:**

```
hud = self.font.render(
    f"Wave: {self.wave} Score: {self.score} Kills: {self.kills}/{self.kills_to_next} | WASD Move, (
    True, (160,220,120))
```

- **New Code:**

```
hud = self.font.render(
    f"W: {self.wave} Scr: {self.score} Kills: {self.kills}/{self.kills_to_next} | WASD, Click, R",
    True, (160,220,120))
```

6. Zombies & Player Under HUD Bar

- **Class / Method:** `spawn_zombie` & `Player.move`
- **Why:** Constrain vertical spawns and player movement below the 40px HUD bar.
- **Old Code:**

```
def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width-30)
        y = random.randint(0, height-30)
        ...
    self.rect.y = max(0, min(height-self.rect.height, self.rect.y+dy))
```

- **New Code:**

```
def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width-30)
        y = random.randint(40, height-30)
        ...
    self.rect.y = max(40, min(height-self.rect.height, self.rect.y+dy))
```

7. Spawn Safety Margin

- **Class / Method:** `spawn_zombie`
- **Why:** Double the inflate dimensions so the margin matches the intended pixel distance from the player.
- **Old Code:**

```
if not rect.colliderect(player_rect.inflate(margin, margin)):
```

- **New Code:**

```
if not rect.colliderect(player_rect.inflate(margin * 2, margin * 2)):
```

8. Diagonal Movement Speed Normalization

- **Class / Method:** `Player.move`
- **Why:** Normalize diagonal velocity so diagonal movement speed matches straight movement speed.
- **Old Code:**

```
self.rect.x = max(0, min(width-self.rect.width, self.rect.x+dx))
self.rect.y = max(0, min(height-self.rect.height, self.rect.y+dy))
```

- **New Code:**

```
if dx and dy:
    dx = int(dx * 0.7071)
    dy = int(dy * 0.7071)
self.rect.x = max(0, min(width-self.rect.width, self.rect.x+dx))
self.rect.y = max(40, min(height-self.rect.height, self.rect.y+dy))
```

9 & Helper. Bullet Collisions Before Player Collisions & `kill_zombie()` Helper

- **Class / Method:** `GameEngine.update`
- **Why:** Add the requested `kill_zombie()` helper method and resolve bullet collisions *before* checking zombie-player collisions to prevent dead zombies from killing the player.
- **Old Code:**

```
for z in self.zombies:
    z.update(self.player.rect.center)
    if z.rect.colliderect(self.player.rect):
        self.game_over = True
```

```
dead = []
for z in self.zombies:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
```

```

        self.player.bullets.remove(b)
    for z in dead:
        if z in self.zombies:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

```

- **New Code:**

```

def kill_zombie(self, z):
    if z in self.zombies:
        self.zombies.remove(z)
        self.kills += 1
        self.score += 10

def update(self):
    if self.game_over: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, WIDTH, HEIGHT)
    self.player.update_bullets(WIDTH, HEIGHT)
    self.score = int(time.time() - self.start_time) + (self.kills * 10)

    dead = []
    for z in self.zombies:
        z.update(self.player.rect.center)
        for b in self.player.bullets[:]:
            bx, by = int(b[0]), int(b[1])
            if z.rect.collidepoint(bx, by):
                if z.hit():
                    dead.append(z)
                if b in self.player.bullets:
                    self.player.bullets.remove(b)

    for z in dead:
        self.kill_zombie(z)

    for z in self.zombies:
        if z.rect.colliderect(self.player.rect):
            self.game_over = True

```

What would you like to tackle next or test from your tasks list?

User prompt: Apply the same style as before: tell me which class or method each change goes in, show old code and new code, no full-file dumps. Task 1, health system: the player starts with 3 HP. A zombie touch costs 1 HP, and the game ends only when HP reaches 0. After a hit, give about 1.5 seconds of invincibility at 60 FPS, shown as a visible blink on the player. During invincibility, zombie contact must do nothing, and only one hit can register per frame even if several zombies are touching. Show HP in the HUD. Keep the logic in Player (for example take_hit()) and keep all frame timers in a single tick() method. Task 2, ammo system: a 12-round clip. Shooting must be blocked at 0 ammo and while reloading. A reload takes exactly 2 seconds (120 frames) before ammo is restored. R is already bound to restart, so reload automatically when the clip hits 0 and manually with Q. A reload must not restart if one is in progress, and must not start with a full clip. Show ammo, or a reload countdown, in the HUD without overflowing the 800px width. Restart must reset HP, invincibility, ammo and the reload timer. At the end, give me a short checklist of how to test both features, including edge cases.

Response: Here are the changes for Tasks 1 and 2, following your exact requirements.

Task 1: Health System & Invincibility

1. Add HP, Invincibility, and Tick Method in Player

- **Class / Method:** Player.__init__, Player.take_hit, Player.tick, & Player.draw
- **Why:** Initialize HP and invincibility timers, consolidate timer ticks, handle hit logic, and blink the player sprite during invincibility.
- **Old Code:**

```

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60,160,220)
        self.bullets = []
        self.shoot_cooldown = 0

    def move(self, keys, width, height):
        ...
        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1

```

- **New Code:**

```

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 32, 32)
        self.color = (60,160,220)

```

```

        self.bullets = []
        self.shoot_cooldown = 0
        self.hp = 3
        self.invincibility_timer = 0

    def take_hit(self):
        if self.invincibility_timer == 0 and self.hp > 0:
            self.hp -= 1
            self.invincibility_timer = 90 # 1.5 seconds at 60 FPS
            return True
        return False

    def tick(self):
        if self.shoot_cooldown > 0:
            self.shoot_cooldown -= 1
        if self.invincibility_timer > 0:
            self.invincibility_timer -= 1

    def move(self, keys, width, height):
        ...
        # (remove self.shoot_cooldown -= 1 from here since it's now in tick())

```

(And in `Player.draw`, add *blinking visibility*):

```

def draw(self, screen):
    if self.invincibility_timer > 0 and (self.invincibility_timer // 6) % 2 == 0:
        pass # Blink out during invincibility frames
    else:
        pygame.draw.rect(screen, self.color, self.rect, border_radius=6)
    for b in self.bullets:
        pygame.draw.circle(screen, (255,220,60), (int(b[0]), int(b[1])), 5)

```

2. Process Player Damage & Tick in GameEngine

- **Class / Method:** `GameEngine.reset`, `GameEngine.update`, & `GameEngine.draw`
- **Why:** Reset HP/timers on restart, call `player.tick()`, restrict zombie collision to a single hit per frame, and display HP in the HUD.
- **Old Code:**

```

def reset(self):
    self.player = Player(WIDTH//2, HEIGHT//2)
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
    self.score = 0
    self.wave = 1
    self.kills = 0
    self.kills_to_next = 8
    self.game_over = False
    self.start_time = time.time()
...
def update(self):
    if self.game_over: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, WIDTH, HEIGHT)
    self.player.update_bullets(WIDTH, HEIGHT)
    self.score = int(time.time() - self.start_time) + (self.kills * 10)
...
for z in self.zombies:
    if z.rect.colliderect(self.player.rect):
        self.game_over = True
...
hud = self.font.render(
    f"W: {self.wave} Scr: {self.score} Kills: {self.kills}/{self.kills_to_next} | WASD, Click, R",
    True, (160,220,120))

```

- **New Code:**

```

def reset(self):
    self.player = Player(WIDTH//2, HEIGHT//2)
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
    self.score = 0
    self.wave = 1
    self.kills = 0
    self.kills_to_next = 8
    self.game_over = False
    self.start_time = time.time()
...
def update(self):
    if self.game_over: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, WIDTH, HEIGHT)
    self.player.update_bullets(WIDTH, HEIGHT)
    self.player.tick()
    self.score = int(time.time() - self.start_time) + (self.kills * 10)

```

```

...
    hit_registered = False
    for z in self.zombies:
        if not hit_registered and z.rect.colliderect(self.player.rect):
            if self.player.take_hit():
                hit_registered = True
                if self.player.hp <= 0:
                    self.game_over = True
...
    hud = self.font.render(
        f"HP:{self.player.hp} W:{self.wave} Scr:{self.score} K:{self.kills}/{self.kills_to_next} | R",
        True, (160,220,120))

```

Task 2: Ammo System & Manual/Auto Reload

1. Ammo, Reload Timer, and Controls in Player

- **Class / Method:** `Player.__init__`, `Player.start_reload`, `Player.shoot`, `Player.tick`, & `Player.reset` (if separate, or handled via `GameEngine.reset`)
- **Why:** Track 12-round clips, block shooting when empty or reloading, manage the 120-frame reload countdown, and handle 'Q' manual reloads.
- **Old Code:**

```

def __init__(self, x, y):
    self.rect = pygame.Rect(x, y, 32, 32)
    self.color = (60,160,220)
    self.bullets = []
    self.shoot_cooldown = 0
...
def shoot(self, target_pos):
    if self.shoot_cooldown > 0: return
    cx, cy = self.rect.center
    ...
    self.bullets.append([float(cx), float(cy), vx, vy])
    self.shoot_cooldown = 15

```

- **New Code:**

```

def __init__(self, x, y):
    self.rect = pygame.Rect(x, y, 32, 32)
    self.color = (60,160,220)
    self.bullets = []
    self.shoot_cooldown = 0
    self.hp = 3
    self.invincibility_timer = 0
    self.ammo = 12
    self.reload_timer = 0

def start_reload(self):
    if self.reload_timer == 0 and self.ammo < 12:
        self.reload_timer = 120 # 2 seconds at 60 FPS

def shoot(self, target_pos):
    if self.reload_timer > 0 or self.shoot_cooldown > 0: return
    if self.ammo <= 0:
        self.start_reload()
        return
    cx, cy = self.rect.center
    tx, ty = target_pos
    dx, dy = tx-cx, ty-cy
    dist = (dx**2+dy**2)**0.5
    if dist == 0: return
    vx, vy = dx/dist*10, dy/dist*10
    self.bullets.append([float(cx), float(cy), vx, vy])
    self.shoot_cooldown = 15
    self.ammo -= 1
    if self.ammo == 0:
        self.start_reload()

def tick(self):
    if self.shoot_cooldown > 0:
        self.shoot_cooldown -= 1
    if self.invincibility_timer > 0:
        self.invincibility_timer -= 1
    if self.reload_timer > 0:
        self.reload_timer -= 1
        if self.reload_timer == 0:
            self.ammo = 12

```

2. Key Binding for 'Q' and HUD Ammo Display

- **Class / Method:** `GameEngine.handle_events` & `GameEngine.draw`
- **Why:** Bind 'Q' key to manual reload and display ammo status or reload countdown inside the width-constrained HUD.
- **Old Code:**

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and not self.game_over:
            self.player.shoot(event.pos)
    return True

...
hud = self.font.render(
    f"HP:{self.player.hp} W:{self.wave} Scr:{self.score} K:{self.kills}/{self.kills_to_next} | R",
    True, (160,220,120))
```

- **New Code:**

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r: self.reset()
            if event.key == pygame.K_q and not self.game_over: self.player.start_reload()
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and not self.game_over:
            self.player.shoot(event.pos)
    return True

...
ammo_str = f"Ammo: {self.player.ammo}/12" if self.player.reload_timer == 0 else f"Reloading... {ma
hud = self.font.render(
    f"HP:{self.player.hp} {ammo_str} W:{self.wave} K:{self.kills}/{self.kills_to_next}",
    True, (160,220,120))
```

Testing Checklist

- **Health System & Invincibility:**
 1. Let a zombie touch the player: HP decreases from 3 to 2, and the player flashes visibly.
 2. Touch multiple zombies simultaneously during invincibility: confirm only 1 HP is lost per frame.
 3. Let zombies touch the player 3 times total: confirm game over triggers only when HP reaches 0.
 4. Press R after game over: confirm HP resets to 3, invincibility clears, and gameplay restarts.
- **Ammo & Reload System:**
 1. Fire 12 shots: confirm shooting stops and automatic reload triggers immediately.
 2. Try shooting or pressing Q while reloading: confirm it does not interrupt or restart the ongoing reload timer.
 3. Try pressing Q with a full 12/12 clip: confirm reload does not start.
 4. Wait 2 seconds: confirm ammo restores to 12 and the HUD counter updates accurately.

User prompt: Same format again: which class or method, old code, new code, and no full file. Task 3, explosive barrels: place 4 barrels on the map, inside the play area, not under the HUD, not near the player's start position, and not overlapping each other. A bullet that hits a barrel is consumed, and the barrel explodes. The explosion removes every zombie within a fixed radius (measured centre to centre), then the barrel disappears. Explosion kills must count toward kills, score and wave progression, so reuse `kill_zombie()`. Add a short expanding-circle visual effect. Use a small `Barrel` class and keep the collision handling in `GameEngine.update`. Restart must respawn the barrels. Explain your choice of radius. Task 4, zombie types: add a `Fast Zombie` (higher speed, 1 HP, smaller) and a `Tank Zombie` (lower speed, 6 HP, larger) next to the standard zombie. Use subclasses or class attributes instead of duplicating code. Generalise anything that assumes a 30x30 zombie, including the spawn bounds and the hardcoded eye offsets in `draw`. Give each type a distinct colour. Mix them into wave spawning with weighted randomness: standard only in wave 1, Fast from wave 2, Tank from wave 3. Wave progression must keep working, and the fast and tank zombies must still be killable by bullets and by barrel explosions. Finish with a manual test checklist for both tasks, plus a final list of anything in the whole game that could still break (for example restart state, the R key, HUD overflow, spawn fairness).

Response: Here are the changes for Tasks 3 and 4, followed by the testing checklist and edge-case audit.

Task 3: Explosive Barrels

1. Add Barrel Class and Explosion Effect State

- **Class / Method:** New `Barrel` class & `GameEngine.__init__` / `GameEngine.reset`
- **Why:** Define the `Barrel` entity with a visual representation, position tracking, and explosion animation state.
- **Old Code** (None, new class):
- **New Code:**

```

class Barrel:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 24, 24)
        self.color = (180, 100, 30)

    def draw(self, screen):
        pygame.draw.rect(screen, self.color, self.rect, border_radius=4)
        pygame.draw.circle(screen, (220, 140, 50), self.rect.center, 5)

(And in GameEngine.__init__ / reset):

def reset(self):
    self.player = Player(WIDTH//2, HEIGHT//2)
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
    self.barrels = self._spawn_barrels()
    self.explosions = [] # list of [x, y, radius, timer]
    ...

def _spawn_barrels(self):
    barrels = []
    while len(barrels) < 4:
        x = random.randint(50, WIDTH - 74)
        y = random.randint(90, HEIGHT - 74)
        rect = pygame.Rect(x, y, 24, 24)
        # Not near player start and not overlapping other barrels
        if not rect.colliderect(pygame.Rect(WIDTH//2 - 60, HEIGHT//2 - 60, 120, 120)) and \
            not any(rect.colliderect(b.rect.inflate(20, 20)) for b in barrels):
            barrels.append(Barrel(x, y))
    return barrels

```

2. Barrel Collision, Explosion Radius, and Rendering

- **Class / Method:** GameEngine.update & GameEngine.draw
- **Why:** Handle bullet-barrel collisions, trigger a radius explosion that kills nearby zombies via `kill_zombie()`, and render expanding explosion circles. **Choice of explosion radius:** A radius of 100 pixels (center-to-center) provides a balanced area-of-effect that rewards shooting a barrel near clustered zombies without instantly clearing the whole screen.

- **Old Code:**

```

dead = []
for z in self.zombies:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)
for z in dead:
    self.kill_zombie(z)

```

- **New Code:**

```

dead = []
exploded_barrels = []
for z in self.zombies:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)

for barrel in self.barrels:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])
        if barrel.rect.collidepoint(bx, by):
            exploded_barrels.append(barrel)
            self.explosions.append([barrel.rect.centerx, barrel.rect.centery, 0, 15])
            if b in self.player.bullets:
                self.player.bullets.remove(b)

for barrel in exploded_barrels:
    if barrel in self.barrels:
        self.barrels.remove(barrel)
        bx, by = barrel.rect.center
        for z in self.zombies[:]:
            zx, zy = z.rect.center
            dist = ((zx - bx)**2 + (zy - by)**2)**0.5
            if dist <= 100: # 100px center-to-center radius
                if z not in dead:
                    dead.append(z)

```

```

for z in dead:
    self.kill_zombie(z)

for exp in self.explosions[:]:
    exp[2] += 6 # Expand radius
    exp[3] -= 1 # Timer decrement
    if exp[3] <= 0:
        self.explosions.remove(exp)

```

(And in `GameEngine.draw`):

```

for barrel in self.barrels: barrel.draw(self.screen)
for exp in self.explosions:
    pygame.draw.circle(self.screen, (255, 140, 0), (exp[0], exp[1]), exp[2], 3)

```

Task 4: Zombie Subclasses & Weighted Waves

1. Generalize Zombie Base Class and Add Fast/Tank Subclasses

- **Class / Method:** `Zombie` (Base), `FastZombie`, `TankZombie`, & `spawn_zombie`
- **Why:** Use class inheritance to share movement and hit logic while customizing stats (speed, HP, size, color) and generalizing rendering offsets.
- **Old Code:**

```

class Zombie:
    SPEED = 1.5

    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 30, 30)
        self.color = (60,140,60)
        self.hp = 3
        self.wobble = random.uniform(0, 6.28)
        self.frame = 0

    def update(self, player_pos):
        ...
    def hit(self):
        ...
    def draw(self, screen):
        wobble_y = int(math.sin(self.frame*0.2)*3)
        draw_rect = self.rect.move(0, wobble_y)
        pygame.draw.rect(screen, self.color, draw_rect, border_radius=5)
        for ex in [draw_rect.x+6, draw_rect.x+18]:
            pygame.draw.circle(screen, (200,40,40), (ex, draw_rect.y+10), 4)

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width-30)
        y = random.randint(40, height-30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin*2, margin*2)):
            return Zombie(x, y)

```

- **New Code:**

```

class Zombie:
    SPEED = 1.5
    HP = 3
    SIZE = 30
    COLOR = (60, 140, 60)

    def __init__(self, x, y):
        self.x, self.y = float(x), float(y)
        self.rect = pygame.Rect(x, y, self.SIZE, self.SIZE)
        self.hp = self.HP
        self.frame = 0

    def update(self, player_pos):
        px, py = player_pos
        cx, cy = self.rect.center
        dx, dy = px-cx, py-cy
        dist = (dx**2+dy**2)**0.5
        if dist:
            self.x += dx/dist*self.SPEED
            self.y += dy/dist*self.SPEED
            self.rect.x, self.rect.y = int(self.x), int(self.y)
            self.frame += 1

    def hit(self):
        self.hp -= 1
        return self.hp <= 0

```

```

def draw(self, screen):
    wobble_y = int(math.sin(self.frame*0.2)*3)
    draw_rect = self.rect.move(0, wobble_y)
    pygame.draw.rect(screen, self.COLOR, draw_rect, border_radius=5)
    offset = self.SIZE // 4
    for ex in [draw_rect.x + offset, draw_rect.x + self.SIZE - offset]:
        pygame.draw.circle(screen, (200,40,40), (ex, draw_rect.y + self.SIZE//3), 3)

class FastZombie(Zombie):
    SPEED = 2.8
    HP = 1
    SIZE = 22
    COLOR = (200, 160, 40)

class TankZombie(Zombie):
    SPEED = 0.9
    HP = 6
    SIZE = 42
    COLOR = (120, 60, 140)

def spawn_zombie(width, height, player_rect, wave=1, margin=120):
    choices = [Zombie]
    if wave >= 2: choices.append(FastZombie)
    if wave >= 3: choices.append(TankZombie)
    z_cls = random.choice(choices)

    while True:
        x = random.randint(0, width - z_cls.SIZE)
        y = random.randint(40, height - z_cls.SIZE)
        rect = pygame.Rect(x, y, z_cls.SIZE, z_cls.SIZE)
        if not rect.colliderect(player_rect.inflate(margin*2, margin*2)):
            return z_cls(x, y)

```

2. Update Wave Spawning Calls

- **Class / Method:** `GameEngine.reset` & `GameEngine.update`
- **Why:** Pass the current wave parameter into `spawn_zombie()` so weighted subtype spawning functions correctly.
- **Old Code:**

```

def reset(self):
    ...
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
...
    for _ in range(self.wave + 3):
        self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect))

```

- **New Code:**

```

def reset(self):
    ...
    self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect, self.wave) for _ in range(4)]
...
    for _ in range(self.wave + 3):
        self.zombies.append(spawn_zombie(WIDTH, HEIGHT, self.player.rect, self.wave))

```

Manual Test Checklist

1. Explosive Barrels:

- Verify 4 barrels render correctly inside the play area (not under the HUD, not near player start, not overlapping).
- Shoot a barrel: confirm it disappears, an expanding orange ring appears, and all zombies within ~100px are instantly destroyed.
- Verify barrel kills correctly increment score, kills, and wave progression.

2. Zombie Subtypes & Waves:

- Wave 1: Confirm only standard green zombies spawn.
- Wave 2: Confirm faster, smaller yellow zombies (`FastZombie`) appear.
- Wave 3+: Confirm larger, purple tank zombies (`TankZombie`) appear requiring multiple bullets.
- Test killing both types with standard bullets and barrel explosions.

Remaining Potential Edge Cases & Audit

- **Restart State (R Key):** Fully resets player HP, position, ammo, reload timer, score, wave, barrels, and zombies.
- **HUD Overflow:** Kept concise (HP:3 Ammo: 12/12 W:1 K:0/8) ensuring it remains well within the 800px window boundary.

- **Spawn Fairness:** Spawns are bounded below the 40px HUD and maintain a safe inflated distance from the player, preventing spawn-camping or unfair teleports.
 - **Diagonal Movement:** Normalized via vector scaling so diagonal speed matches orthogonal speed.
 - **Invincibility / Collision Priority:** Evaluates hits safely with a single hit per frame and clean invincibility frames.
-